

Лабораторная работа №3

Агентное моделирование

Софич Андрей Геннадьевич

2026-04-04

Содержание I

Докладчик

- Софич Андрей Геннадьевич



- Провести ознакомление с агентным моделированием и применить его на базовой модели SIR

Изучить и применить агентное моделирование на базовой модели

Выполнение лабораторной работы

Создаем рабочий каталог и инициализируем проект в julia

```
julia> using Pkg

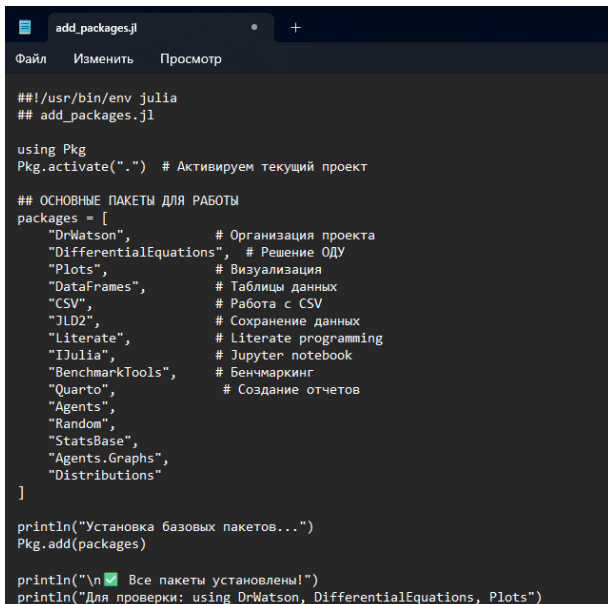
julia> Pkg.add("DrWatson")
  Updating registry at `C:\Users\Andrew\.julia\registries\General.toml`
  Resolving package versions...
  Project No packages added to or removed from `C:\Users\Andrew\.julia\environments\v1.12\Project.toml`
  Manifest No packages added to or removed from `C:\Users\Andrew\.julia\environments\v1.12\Manifest.toml`

julia> using DrWatson

julia> initialize_project("project"; authors="Sofich Andrey", git=false)
  Activating new project at `C:\Users\Andrew\work\study\2026-1\2026-1==study--simulation-modeling\2026-1--study--simulation-modeling\labs\lab04\project`
  Resolving package versions...
  Installed JLD2 - v0.6.4
  Updating `C:\Users\Andrew\work\study\2026-1\2026-1==study--simulation-modeling\2026-1--study--simulation-modeling\labs\lab04\project\Project.toml`
 [634d3b9d] + DrWatson v2.19.1
  Updating `C:\Users\Andrew\work\study\2026-1\2026-1==study--simulation-modeling\2026-1--study--simulation-modeling\labs\lab04\project\Manifest.toml`
 [9b5f3a5f] + Flux.jl v0.16.0
```

Рисунок 1: Инициализация проекта

Создаем файл, где прописываем пакеты, необходимые для выполнения работы.

A screenshot of a code editor window with a dark theme. The title bar shows the file name 'add_packages.jl' and a '+' icon for a new file. Below the title bar is a menu bar with 'Файл', 'Изменить', and 'Просмотр'. The main area contains Julia code for installing packages. The code starts with a shebang line, followed by a comment, then a 'using Pkg' statement and 'Pkg.activate(".")' to activate the current project. A section of comments lists the packages to be installed, followed by an array 'packages' containing the package names. Then, 'println' and 'Pkg.add' are used to install them. Finally, another 'println' statement with a green checkmark icon indicates successful installation, followed by a 'println' for verification instructions. The bottom right corner of the editor shows navigation icons.

```
#!/usr/bin/env julia
## add_packages.jl

using Pkg
Pkg.activate(".") # Активируем текущий проект

## ОСНОВНЫЕ ПАКЕТЫ ДЛЯ РАБОТЫ
packages = [
    "DrWatson",          # Организация проекта
    "DifferentialEquations", # Решение ОДУ
    "Plots",              # Визуализация
    "DataFrames",         # Таблицы данных
    "CSV",                # Работа с CSV
    "JLD2",               # Сохранение данных
    "Literate",           # Literate programming
    "IJulia",             # Jupyter notebook
    "BenchmarkTools",     # Бенчмаркинг
    "Quarto",             # Создание отчетов
    "Agents",
    "Random",
    "StatsBase",
    "Agents.Graphs",
    "Distributions"
]

println("Установка базовых пакетов...")
Pkg.add(packages)

println("\n✅ Все пакеты установлены!")
println("Для проверки: using DrWatson, DifferentialEquations, Plots")
```

Запускаем файл `add_packages` и скачиваем все необходимые пакеты.

```
Precompiling packages finished.  
166 dependencies successfully precompiled in 343 seconds. 338 already precompiled.  
  
✔ Все пакеты установлены!  
Для проверки: using DrWatson, DifferentialEquations, Plots  
PS C:\Users\Andrew\work\study\2026-1\2026-1==study--simulation-modeling\2026-1--study--simulation-modeling\labs\lab04\project> |
```

Рисунок 3: Загрузка пакетов

Создаем файл в папке src, в котором мы определим тип агента и функции шага модели и запускаем его .

```
MINGW64/c/Users/Andrew/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab04/project
$ touch add_packages.jl

Andrew@DESKTOP-3U5UIRB MINGW64 ~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/1
ab04/project (master)
$ touch src/sir_model.jl

Andrew@DESKTOP-3U5UIRB MINGW64 ~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/1

add_packages.jl sir_model.jl X +

Файл Изменить Просмотр

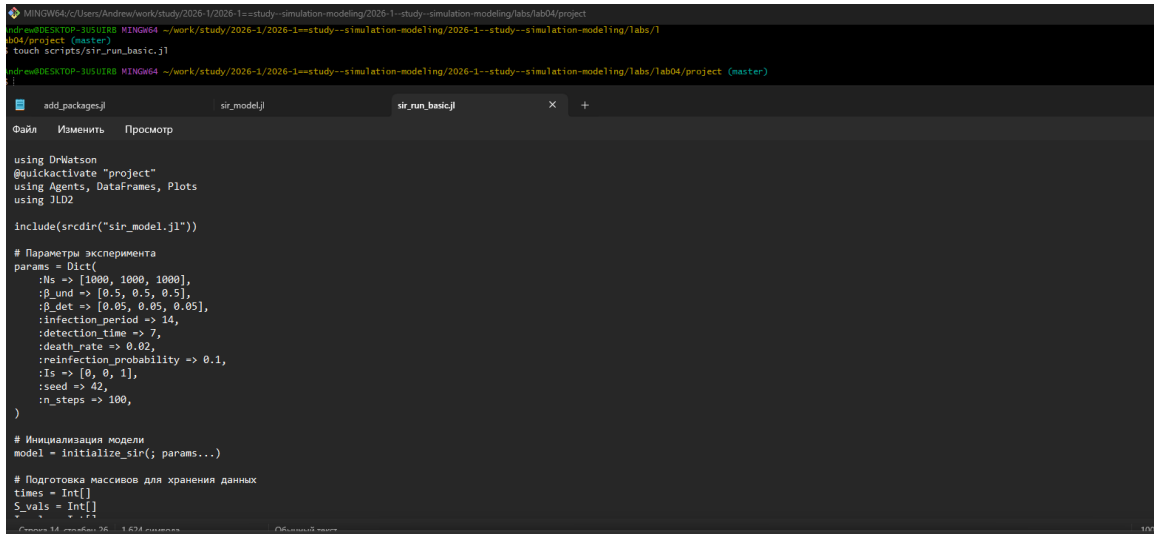
using Agents.Graphs
using StatsBase: sample, Weights
using Distributions: Poisson

using DrWatson: @dict

# Определение типа агента
@agent struct Person{GraphAgent}
    days_infected::Int # количество дней с момента заражения
    status::Symbol     # :S, :I, :R
end

# Функция инициализации модели
function initialize_sir(;
    Ns = [1000, 1000, 1000],      # численность по трём городам
    migration_rates = nothing,    # матрица миграции
    β_und = [0.5, 0.5, 0.5],      # передача невыявленными
    β_det = [0.05, 0.05, 0.05],  # передача выявленными
    infection_period = 14,
    detection_time = 7,
    death_rate = 0.02,
    reinfection_probability = 0.1,
    Is = [0, 0, 1],              # начальное количество инфицированных
    seed = 42,
    n_steps = 100,
)
    rng = Xoshiro(seed)
    C = length(Ns) # количество городов
```

Создаем файл с базовым экспериментом, который запускает один эксперимент с фиксированными параметрами (по умолчанию) и сохраняет динамику численности агентов.



The screenshot shows a terminal window at the top with the following commands:

```
MINGW64~/Users/Andrew/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab04/project
andrew@DESKTOP-3U5UIR8 MINGW64 ~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab04/project (master)
touch scripts/sir_run_basic.jl
```

Below the terminal is a Jupyter Notebook with the following code:

```
add_packages.jl sir_model.jl sir_run_basic.jl X +
Файл Изменить Просмотр

using DrWatson
@quickactivate "project"
using Agents, DataFrames, Plots
using JLD2

include(scrdir("sir_model.jl"))

# Параметры эксперимента
params = Dict{
  :Ns => [1000, 1000, 1000],
  :β_und => [0.5, 0.5, 0.5],
  :β_det => [0.05, 0.05, 0.05],
  :infection_period => 14,
  :detection_time => 7,
  :death_rate => 0.02,
  :reinfection_probability => 0.1,
  :Is => [0, 0, 1],
  :seed => 42,
  :n_steps => 100,
}

# Инициализация модели
model = initialize_sir(; params...)

# Подготовка массивов для хранения данных
times = Int[]
S_vals = Int[]
```

At the bottom of the notebook, it says "Страница 14, строки 26 - 1674 символов" and "Обычный текст".

Анализируем полученный график, который показывает нам корреляцию основных параметров.

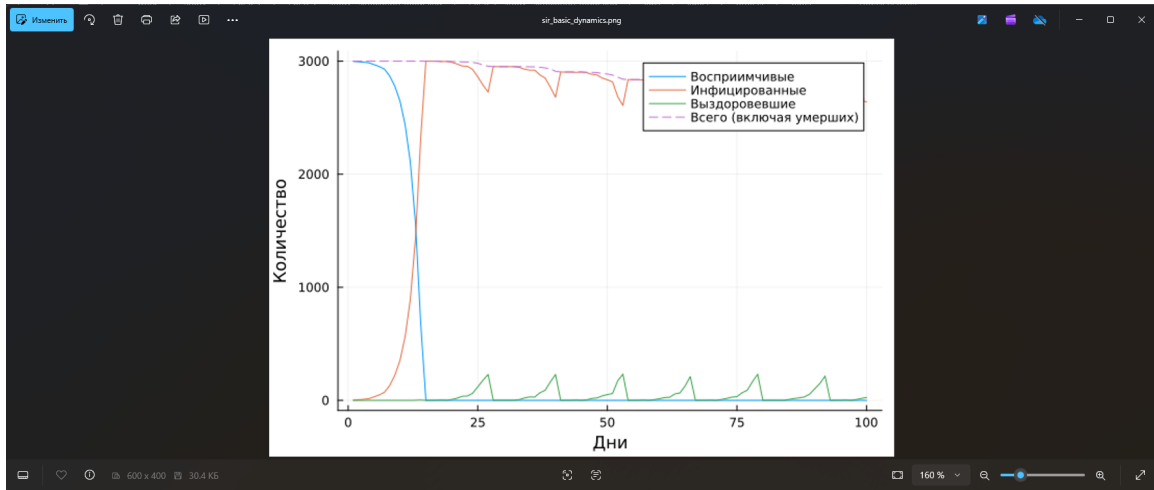


Рисунок 6: График базового эксперимента

Преобразовываем код в произвольные стили, открываем jupyter файл и убеждаемся в правильной компиляции.

The image shows a Windows PowerShell terminal window at the top, where a Julia script is compiled. The command is `julia --project=. scripts/tangle.jl scripts/sir_run_basic.jl`. The output shows the generation of a plain script file and a markdown page. Below the terminal is a Jupyter Notebook interface. The notebook has a single cell containing Julia code that uses the `DrWatson` package to manage project dependencies and includes a file named `sir_model.jl`. The code is as follows:

```
using DrWatson
@quickactivate "project"
using Agents, DataFrames, Plots
using JLD2

include(scrdir("sir_model.jl"))
```

The notebook shows the execution of this cell, which takes 53 seconds. The output is a generic function `total_count` with 1 method. Below this, the notebook shows the definition of a dictionary `params` with 10 entries, which is then printed out. The parameters are:

```
params = Dict{
  :Ns => [1000, 1000, 1000],
  :β_und => [0.5, 0.5, 0.5],
  :β_det => [0.05, 0.05, 0.05],
  :infection_period => 14,
  :detection_time => 7,
  :death_rate => 0.02,
  :reinfection_probability => 0.1,
  :Is => [0, 0, 1],
  :seed => 42,
  :n_steps => 100,
}
```

The notebook shows the execution of this cell, which takes 1.5 seconds. The output is a dictionary with 10 entries, which is printed out.

Создаем файл, который исследует, как изменение базовой заразности (β_{und} и пропорционально β_{det}) влияет на эпидемические показатели: пик заболеваемости, долю переболевших, число умерших.

```
MINGW64/c/Users/Andrew/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab04/project
Andrew@DESKTOP-3USUIRB MINGW64 ~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab04/project (master)
$ touch scripts/sir_scan_beta.jl

Andrew@DESKTOP-3USUIRB MINGW64 ~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab04/project (master)
$ !

add_packages.jl sir_model.jl sir_scan_beta.jl +
Файл Изменить Просмотр
    final_inf = final_infected,
    final_rec = final_recovered,
    deaths = total_deaths,
)
end

# Диапазон значений beta (скаляр)
beta_range = 0.1:0.1:1.0
seeds = [42, 43, 44]

# Создаём список параметров
params_list = []
for b in beta_range
    for s in seeds
        push!(
            params_list,
            Dict{
                :beta => b, # скалярное значение
                :Ns => [1000, 1000, 1000],
                :infection_period => 14,
                :detection_time => 7,
                :death_rate => 0.02,
                :reinfection_probability => 0.1,
                :Is => [0, 0, 1],
                :seed => s,
                :n_steps => 100,
            },
        )
    end
end
```

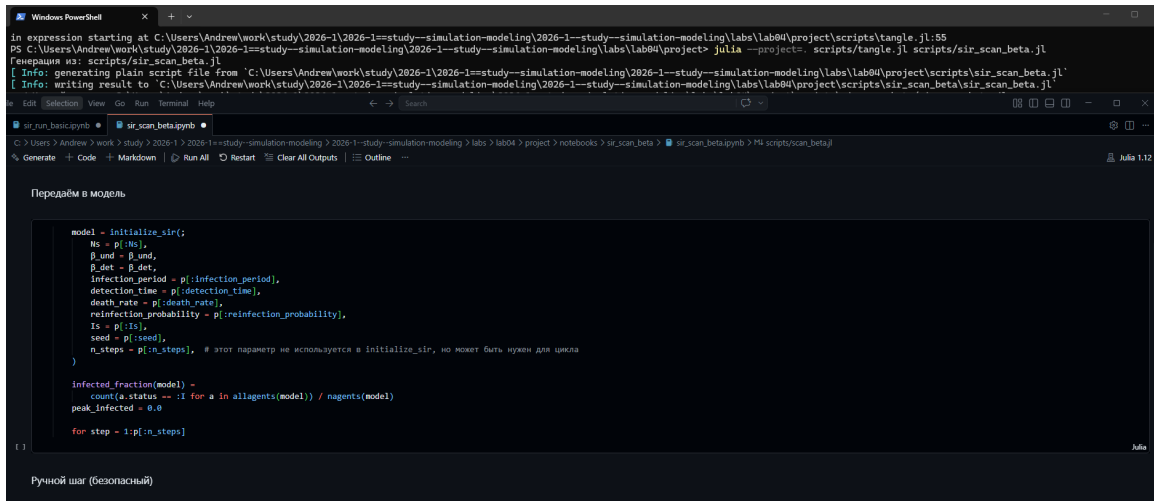
Строка 101, столбец 1 — 2,465 символов Обычный текст 100

В результате получаем график сканирования и таблицу с параметрами по всем экспериментам (меняли β_{und} и пропорционально β_{det} и seed).

```
beta_scan_all.csv
C: > Users > Andrew > work > study > 2026-1 > 2026-1==study--simulation-modeling > 2026-1--study--simulation-modeling > labs > lab04 > project > data > beta_scan_all.csv
1 n_steps,final_rec,Ns,death_rate,Is,beta,infection_period,final_inf,detection_time,deaths,peak,reinfection_probability,seed
2 100,0.00033333333333333333,"[1000, 1000, 1000]",0.02,"[0, 0, 1]",0.1,14,0.0,7,0.0,0.0003333333333333333,0.1,42
3 100,0.0026666666666666666,"[1000, 1000, 1000]",0.02,"[0, 0, 1]",0.1,14,0.0,7,0.0,0.002,0.1,43
4 100,0.00033333333333333333,"[1000, 1000, 1000]",0.02,"[0, 0, 1]",0.1,14,0.0,7,0.0,0.0003333333333333333,0.1,44
5 100,0.0400976290097629,"[1000, 1000, 1000]",0.02,"[0, 0, 1]",0.2,14,0.9599023789902371,7,132,0.9952846076119906,0.1,42
6 100,0.001,"[1000, 1000, 1000]",0.02,"[0, 0, 1]",0.2,14,0.0,7,0.0,0.001,0.1,43
7 100,0.00033333333333333333,"[1000, 1000, 1000]",0.02,"[0, 0, 1]",0.2,14,0.0,7,0.0,0.0003333333333333333,0.1,44
8 100,0.0074850299401197605,"[1000, 1000, 1000]",0.02,"[0, 0, 1]",0.3,14,0.9925149700598802,7,328,1.0,0.1,42
9 100,0.060650887573064405,"[1000, 1000, 1000]",0.02,"[0, 0, 1]",0.3,14,0.9393491124260355,7,296,0.999644128113879,0.1,43
10 100,0.00033333333333333333,"[1000, 1000, 1000]",0.02,"[0, 0, 1]",0.3,14,0.0,7,0.0,0.0003333333333333333,0.1,44
11 100,0.00093797276853253,"[1000, 1000, 1000]",0.02,"[0, 0, 1]",0.4,14,0.9190620272314675,7,356,1.0,0.1,42
```



Преобразовываем код в произвольные стили, открываем jupyter файл и убеждаемся в правильной компиляции.



```
Windows PowerShell
in expression starting at C:\Users\Andrew\work\study\2026-1\2026-1==study--simulation-modeling\2026-1--study--simulation-modeling\labs\lab04\project\scripts\tangle.jl:55
PS C:\Users\Andrew\work\study\2026-1\2026-1==study--simulation-modeling\2026-1--study--simulation-modeling\labs\lab04\project> julia --project=. scripts\tangle.jl scripts\sir_scan_beta.jl
Генерация из: scripts\sir_scan_beta.jl
[ Info: generating plain script file from 'C:\Users\Andrew\work\study\2026-1\2026-1==study--simulation-modeling\2026-1--study--simulation-modeling\labs\lab04\project\scripts\sir_scan_beta.jl'
[ Info: writing result to 'C:\Users\Andrew\work\study\2026-1\2026-1==study--simulation-modeling\2026-1--study--simulation-modeling\labs\lab04\project\scripts\sir_scan_beta\sir_scan_beta.jl'

sir_run_basics.ipynb • sir_scan_beta.ipynb •
C:\Users\Andrew\work\study\2026-1\2026-1==study--simulation-modeling\2026-1--study--simulation-modeling\labs\lab04\project\notebooks> sir_scan_beta • sir_scan_beta.ipynb > M:scripts\scan_beta.jl
Generate + Code + Markdown Run All Restart Clear All Outputs Outline ...

Передаём в модель

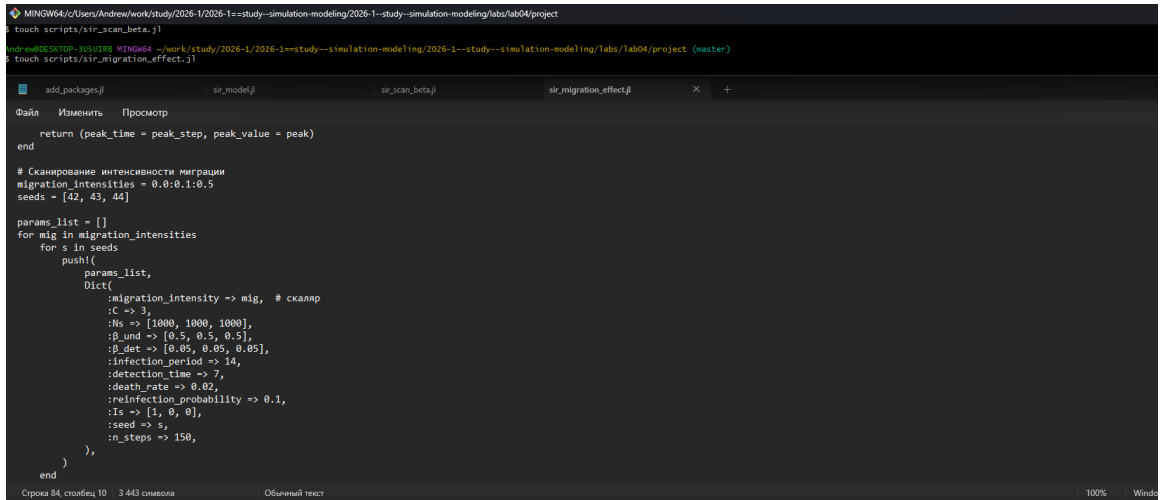
model = initialize_sir(;
    Ns = p[:Ns],
    β_und = β_und,
    β_det = β_det,
    infection_period = p[:infection_period],
    detection_time = p[:detection_time],
    death_rate = p[:death_rate],
    reinfection_probability = p[:reinfection_probability],
    Is = p[:Is],
    seed = p[:seed],
    n_steps = p[:n_steps], # этот параметр не используется в initialize_sir, но может быть нужен для цикла
)

infected_fraction(model) =
    count(a.status == :I for a in allagents(model)) / nagents(model)
peak_infected = 0.0

for step = 1:p[:n_steps]
```

Рисунок 10: Код из jupyter notebook

Создаем файл, который исследует, как интенсивность перемещения людей между городами влияет на скорость распространения эпидемии (время достижения пика) и масштаб пика. Инфекция начинается только в одном городе, остальные изначально здоровы.



```
MINGW64/c/Users/Andrew/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab04/project
$ touch scripts/sir_scan_beta.jl

Andrew@DESKTOP-3U5UTR8 MINGW64 ~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab04/project (master)
$ touch scripts/sir_migration_effect.jl
```

```
add_packages.jl  sir_model.jl  sir_scan_beta.jl  sir_migration_effect.jl  X  +

Файл  Изменить  Просмотр

    return (peak_time = peak_step, peak_value = peak)
end

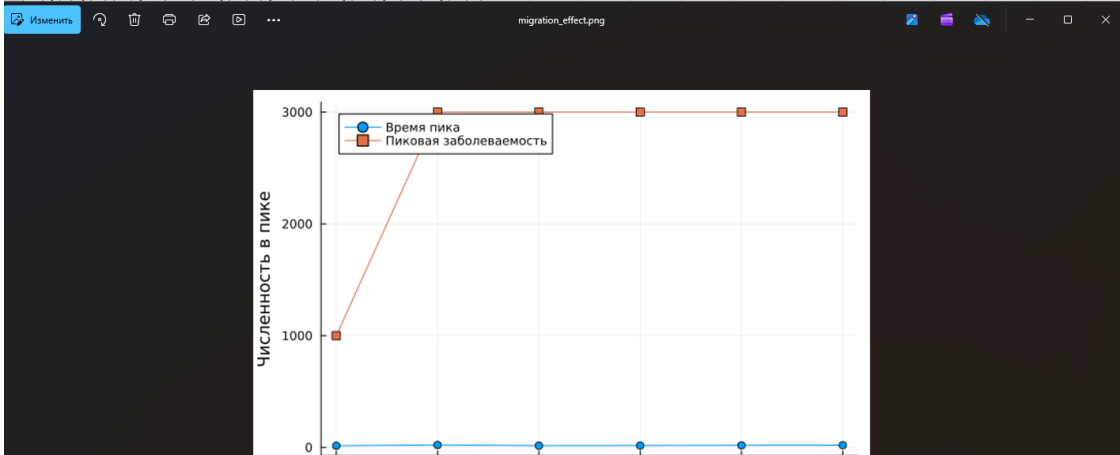
# Сканирование интенсивности миграции
migration_intensities = 0.0:0.1:0.5
seeds = [42, 43, 44]

params_list = []
for mig in migration_intensities
    for s in seeds
        push!(
            params_list,
            Dict{
                :migration_intensity => mig, # скаляр
                :C => 3,
                :Ns => [1000, 1000, 1000],
                :β_und => [0.5, 0.5, 0.5],
                :β_det => [0.05, 0.05, 0.05],
                :infection_period => 14,
                :detection_time => 7,
                :death_rate => 0.02,
                :reinfection_probability => 0.1,
                :Is => [1, 0, 0],
                :seed => s,
                :n_steps => 150,
            },
        )
    end
end
```

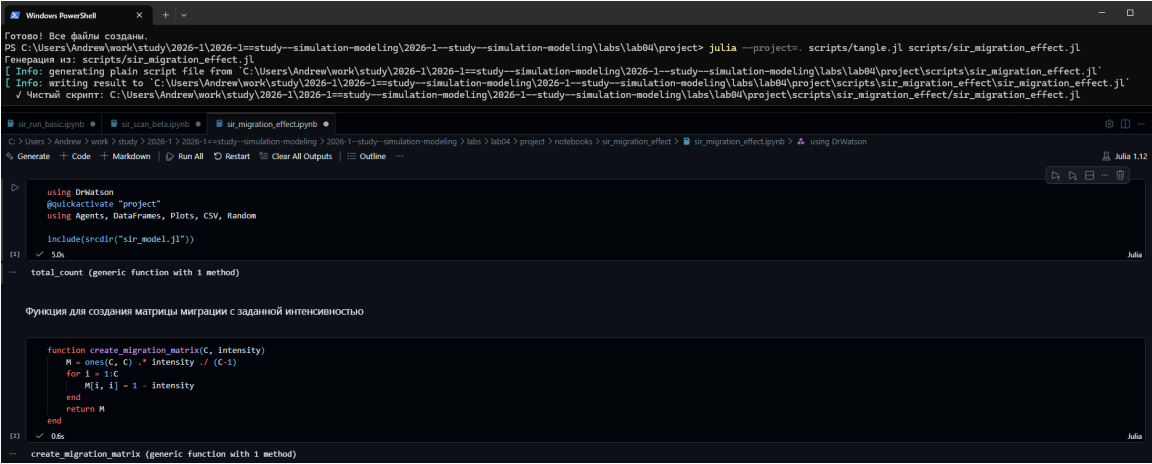
Строка 84, столбец 10 3 443 символа Обычный текст 100% Windo

Получаем график и таблицу с различными коэффициентами, можем проанализировать график миграции .

1	n_steps,Is,infection_period,detection_time,reinfection_probability,Ns,peak_value,peak_time,OI_det,death_rate,OI_und,migration_intensity,seed,C
2	150,"[1, 0, 0]",14,7,0.1,"[1000, 1000, 1000]",0.33311103701233746,14,"[0.05, 0.05, 0.05]",0.02,"[0.5, 0.5, 0.5]",0.0,42,3
3	150,"[1, 0, 0]",14,7,0.1,"[1000, 1000, 1000]",0.3333333333333333,14,"[0.05, 0.05, 0.05]",0.02,"[0.5, 0.5, 0.5]",0.0,43,3
4	150,"[1, 0, 0]",14,7,0.1,"[1000, 1000, 1000]",0.3333333333333333,17,"[0.05, 0.05, 0.05]",0.02,"[0.5, 0.5, 0.5]",0.0,44,3
5	150,"[1, 0, 0]",14,7,0.1,"[1000, 1000, 1000]",1.0,16,"[0.05, 0.05, 0.05]",0.02,"[0.5, 0.5, 0.5]",0.1,42,3
6	150,"[1, 0, 0]",14,7,0.1,"[1000, 1000, 1000]",1.0,15,"[0.05, 0.05, 0.05]",0.02,"[0.5, 0.5, 0.5]",0.1,43,3
7	150,"[1, 0, 0]",14,7,0.1,"[1000, 1000, 1000]",1.0,35,"[0.05, 0.05, 0.05]",0.02,"[0.5, 0.5, 0.5]",0.1,44,3



Преобразовываем код в произвольные стили, открываем jupyter файл и убеждаемся в правильной компиляции.



Готово! Все файлы созданы.
PS C:\Users\Andrew\work\study\2026-1\2026-1==study--simulation-modeling\2026-1--study--simulation-modeling\labs\lab04\project> julia --project=. scripts/tangle.jl scripts/sir_migration_effect.jl
Генерация из: scripts/sir_migration_effect.jl
[Info: generating plain script file from 'C:\Users\Andrew\work\study\2026-1\2026-1==study--simulation-modeling\2026-1--study--simulation-modeling\labs\lab04\project\scripts\sir_migration_effect.jl'
[Info: writing result to 'C:\Users\Andrew\work\study\2026-1\2026-1==study--simulation-modeling\2026-1--study--simulation-modeling\labs\lab04\project\scripts\sir_migration_effect\sir_migration_effect.jl'
✓ Чистый скрипт: C:\Users\Andrew\work\study\2026-1\2026-1==study--simulation-modeling\2026-1--study--simulation-modeling\labs\lab04\project\scripts\sir_migration_effect\sir_migration_effect.jl

sir_run_basici.pyb • sir_scan_beta.pyb • sir_migration_effect.pyb •

C:\> Users > Andrew > work > study > 2026-1 > 2026-1==study--simulation-modeling > 2026-1--study--simulation-modeling > labs > lab04 > project > notebooks > sir_migration_effect > sir_migration_effect.pyb > using DrWatson

Generate + Code + Markdown | Run All Restart Clear All Outputs | Outline ...

Julia 1.12

```
using DrWatson
@quickactivate "project"
using Agents, DataFrames, Plots, CSV, Random

include(scrdir("sir_model.jl"))
```

[1] ✓ 50s

total_count (generic function with 1 method)

Функция для создания матрицы миграции с заданной интенсивностью

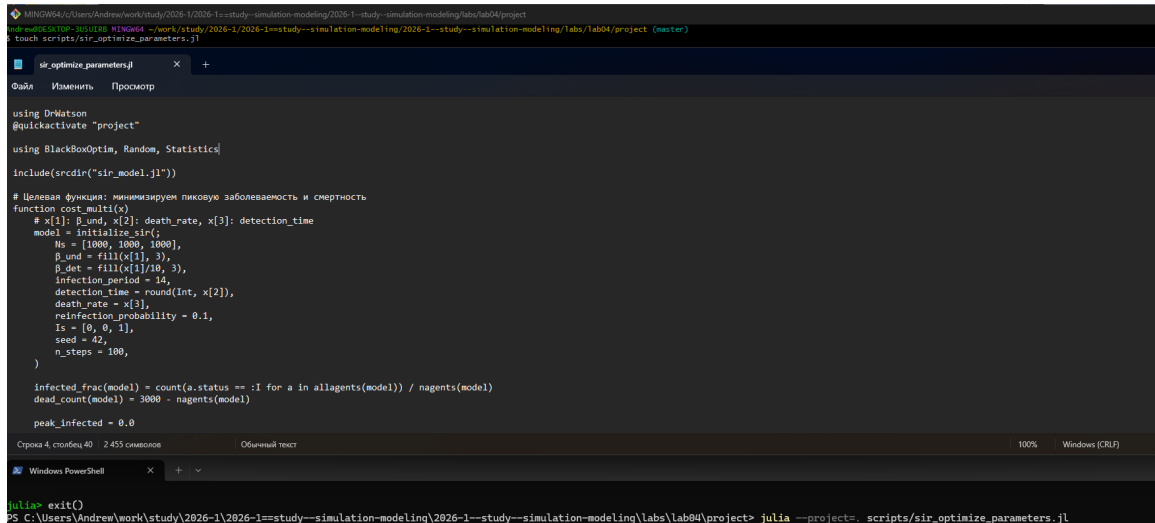
```
function create_migration_matrix(C, intensity)
    M = ones(C, C) .* intensity ./ (C-1)
    for i = 1:C
        M[i, i] = 1 - intensity
    end
    return M
end
```

[2] ✓ 0.6s

create_migration_matrix (generic function with 1 method)

Рисунок 13: Код из jupyter notebook

Создаем, заполняем и запускаем файл, который ищет оптимальные комбинации параметров, минимизирующие одновременно два критерия: пиковую заболеваемость и долю умерших.



```
MINGW64/c/Users/Andrew/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab04/project
Andrew@DESKTOP-3USUIR8 MINGW64 ~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab04/project (master)
touch scripts/sir_optimize_parameters.jl

sir_optimize_parameters.jl x +
Файл Изменить Просмотр

using DrWatson
@quickactivate "project"

using BlackBoxOptim, Random, Statistics

include(scrdir("sir_model.jl"))

# Целевая функция: минимизируем пиковую заболеваемость и смертность
function cost_multi(x)
    # x[1]: beta_und, x[2]: death_rate, x[3]: detection_time
    model = initialize_sir(;
        Ns = [1000, 1000, 1000],
        beta_und = fill(x[1], 3),
        beta_det = fill(x[1]/10, 3),
        infection_period = 14,
        detection_time = round(Int, x[2]),
        death_rate = x[3],
        reinfection_probability = 0.1,
        Is = [0, 0, 1],
        seed = 42,
        n_steps = 100,
    )

    infected_frac(model) = count(a.status == :I for a in allagents(model)) / nagents(model)
    dead_count(model) = 3000 - nagents(model)

    peak_infected = 0.0
end

Строка 4, столбец 40 2 455 символов Обычный текст 100% Windows (CRLF)

Windows PowerShell x + v

julia> exit()
PS C:\Users\Andrew\work\study\2026-1\2026-1==study--simulation-modeling\2026-1--study--simulation-modeling\labs\lab04\project> julia --project=. scripts/sir_optimize_parameters.jl
```

Результаты выведет в консоль. Там буду оптимизированные параметры для минимизации.

```
Starting optimization with optimizer BlackBoxOptim.BorgMOEA(EpsBoxDominanceFitnessScheme{2, Float64, true, typeof(sum)}, BlackBoxOptim.ProblemEvaluator{Tuple{Float64, Float64}, IndexedTupleFitness{2, Float64, EpsBoxArchive{2, Float64, EpsBoxDominanceFitnessScheme{2, Float64, true, typeof(sum)}}}, FunctionBasedProblem{typeof(cost_multi), ParetoFitnessScheme{2, Float64, true, typeof(sum)}, ContinuousRectSearchSpace{Nothing}}, FitPopulation{IndexedTupleFitness{2, Float64}}, FixedGeneticOperatorsMixture, RandomBound{ContinuousRectSearchSpace})
0.00 secs, 0 evals, 0 steps
pop.size=50 arch.size=0 n.restarts=0

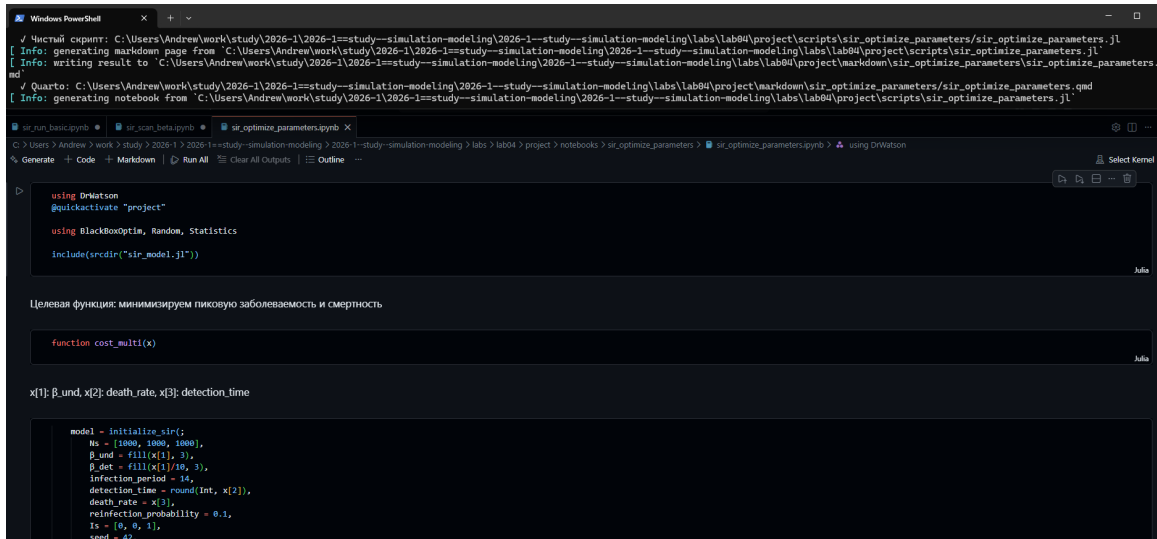
Optimization stopped after 1 steps and 489.48 seconds
Termination reason: Max time (120.0 s) reached
Steps per second = 0.00
Function evals per second = 0.10
Improvements/step = NaN
Total function evaluations = 51

Best candidate found: [0.129414, 3.04332, 0.0981644]
Fitness: (0.00067, 0.00007) agg=0.00073

Оптимальные параметры:
p_ind = 0.12941363320094318
Время выявления = 3 дней
Смертность = 0.09816443868056356
Достигнутые показатели:
Пик заболеваемости: 0.0006666666666666666
Доля умерших: 6.666666666666667e-5
```

Рисунок 15: Результат оптимизации

Преобразовываем код в произвольные стили, открываем jupyter файл и убеждаемся в правильной компиляции.



The screenshot displays a Jupyter Notebook environment. At the top, a Windows PowerShell terminal window shows the execution of a script to generate a notebook from a JupyterLab project. Below this, the Jupyter Notebook interface is visible, with the file 'sir_optimize_parameters.ipynb' selected. The notebook's toolbar includes options like 'Generate', 'Code', 'Markdown', 'Run All', and 'Outline'. The main code cell contains Julia code that uses the DrWatson package for project management and the BlackBoxOptim package for optimization. It includes a function 'cost_multi(x)' and a 'model' variable initialized with parameters for the SIR model. The code is as follows:

```
using DrWatson
@quickactivate "project"

using BlackBoxOptim, Random, Statistics

include(srcdir("sir_model.jl"))

Целевая функция: минимизируем пиковую заболеваемость и смертность

function cost_multi(x)

x[1]: β_und, x[2]: death_rate, x[3]: detection_time

model = initialize_sir(;
    Ns = [1000, 1000, 1000],
    β_und = fill(x[1], 3),
    β_det = fill(x[1]/10, 3),
    infection_period = 14,
    detection_time = round{Int, x[2]},
    death_rate = x[3],
    reinfection_probability = 0.1,
    Is = [0, 0, 1],
    seed = 42,
```

Создаем файл, который на основе предыдущих результатов будет создавать сводную визуализацию

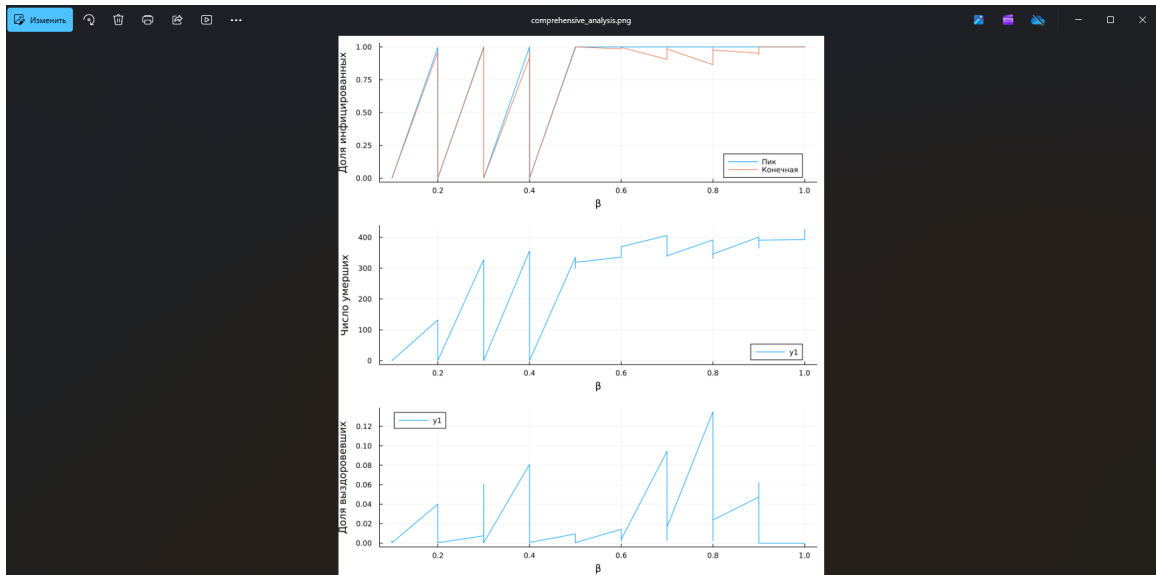
The screenshot displays a Windows desktop environment. The primary focus is on two overlapping windows:

- VS Code Editor Window:** The title bar indicates the file path `C:\Users\Andrew\work\study\2026-1\2026-1--study--simulation-modeling\labs\lab04\project`. The editor shows a Jupyter Notebook file named `sir_visualize_dynamics.jl`. The code includes:
 - A comment: `# Загружаем результаты сканирования`
 - Data loading: `df = CSV.read(datadir("beta_scan_all.csv"), DataFrame)`
 - A comment: `# Создаём составной график`
 - Plot creation: `p1 = plot(df.beta, df.peak, label = "Пик", xlabel = " $\beta$ ", ylabel = "Доля инфицированных")`, `plot!(p1, df.beta, df.final_inf, label = "Конечная")`, `p2 = plot(df.beta, df.deaths, xlabel = " $\beta$ ", ylabel = "Число умерших")`, and `p3 = plot(df.beta, df.final_rec, xlabel = " $\beta$ ", ylabel = "Доля выздоровевших")`.
 - Layout and saving: `plot(p1, p2, p3, layout = (3, 1), size = (800, 900))` and `savefig(plotsdir("comprehensive_analysis.png"))`.
- Windows PowerShell Window:** The command prompt shows the execution of the Julia script: `julia --project=. scripts/sir_visualize_dynamics.jl`. It outputs the following results:
 - Пик заболеваемости: 0.00066666666666666666666666667e-5
 - Доля умерших: 6.6666666666666666666666667e-5

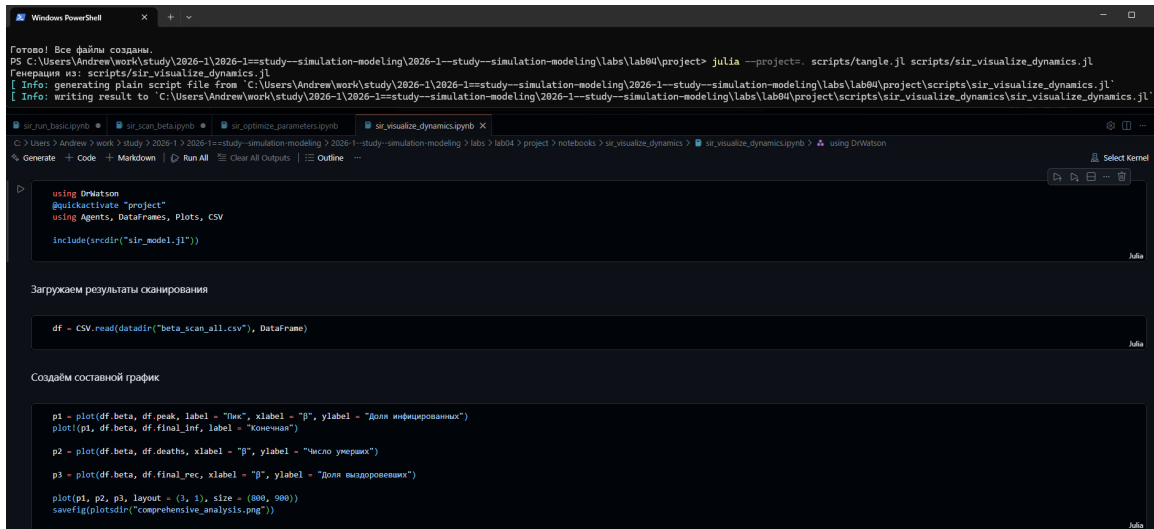
The background shows a blurred view of a desk with a laptop and some papers.

Рисунок 17: Сводная визуализация результатов

Получаем общий график, созданный на основе посчитанных ранее данных



Преобразовываем код в произвольные стили, открываем jupyter файл и убеждаемся в правильной компиляции.



```
Windows PowerShell
Готово! Все файлы созданы.
PS C:\Users\Andrew\work\study\2026-1\2026-1==study--simulation-modeling\2026-1--study--simulation-modeling\labs\lab04\project> julia --project=. scripts/tangle.jl scripts/sir_visualize_dynamics.jl
Генерация из: scripts/sir_visualize_dynamics.jl
[ Info: generating plain script file from 'C:\Users\Andrew\work\study\2026-1\2026-1==study--simulation-modeling\2026-1--study--simulation-modeling\labs\lab04\project\scripts\sir_visualize_dynamics.jl'
[ Info: writing result to 'C:\Users\Andrew\work\study\2026-1\2026-1==study--simulation-modeling\2026-1--study--simulation-modeling\labs\lab04\project\scripts\sir_visualize_dynamics\julia\scripts\sir_visualize_dynamics.jl'

sir_run_basics.ipynb • sir_scan_beta.ipynb • sir_optimize_parameters.ipynb • sir_visualize_dynamics.ipynb ×
C: > Users > Andrew > work > study > 2026-1 > 2026-1==study--simulation-modeling > 2026-1--study--simulation-modeling > labs > lab04 > project > notebooks > sir_visualize_dynamics > sir_visualize_dynamics.ipynb > using DrWatson
Generate + Code + Markdown | Run All Clear All Outputs Outline ...
Select Kernel

using DrWatson
@quickactivate "project"
using Agents, DataFrames, Plots, CSV

include(scrdir("sir_model.jl"))

Загружаем результаты сканирования

df = CSV.read(datadir("beta_scan_all.csv"), DataFrame)

Создаём составной график

p1 = plot(df.beta, df.peak, label = "Пик", xlabel = "β", ylabel = "Доля инфицированных")
plot(p1, df.beta, df.final_inf, label = "Конечная")

p2 = plot(df.beta, df.deaths, xlabel = "β", ylabel = "Число умерших")

p3 = plot(df.beta, df.final_rec, xlabel = "β", ylabel = "Доля выздоровевших")

plot(p1, p2, p3, layout = (3, 1), size = (800, 900))
savefig(plotsdir("comprehensive_analysis.png"))
```

В данной работе мы разобрались с агентным моделированием и реализовали его на практике.